

University of ———

Undergraduate Program of Study
Supervising professor: ———

Thesis
Implementation of a digital system utilising microcontrollers
and logic gates
(GRAD Project)

——— mpaxlamitsounas@mailbox.org ———

———
26/02/2026

Table of contents

Table of contents.....	1
Introduction.....	2
GRAD Machine.....	3
Overview.....	3
Usage.....	3
Functionality.....	4
Memory.....	4
Arithmetic Logic Unit.....	4
ALU Decoder.....	6
JMP Decoder.....	6
Program Counter.....	6
Input Decoders.....	6
Registers.....	6
Instruction Decoder.....	6
Instruction definitions.....	6
Implementation details.....	8
Instruction decoder.....	8
Memory.....	8
Arithmetic Logic Unit.....	8
JMP Decoder.....	8
Program Counter.....	8
Input Decoder.....	9
Registers.....	9
Usage of sources.....	9
The Elements of Computing Systems.....	9
Building Math Circuits.....	10
GRAD Language.....	11
Overview.....	11
Grammar.....	11
Programming in GRAD Language.....	11
Usage of sources.....	13
The Elements of Computing Systems.....	13
GRAD Compiler.....	14
Overview.....	14
Usage.....	14
Compilation process.....	14
Implementation details.....	15
Parser.....	15
Assembler.....	15
Compiler.....	15
Decompressing C-instructions.....	15
Sources.....	18

Introduction

The GRAD Project is a set of software and specifications for a CPU (GRAD Machine), a programming language (GRAD Language), and a compiler and assembler (GRAD Compiler). The architecture of the CPU, as well as the syntax of the programming language were initially directed by the book "The Elements of Computing Systems"[\[1\]](#). However, the project has moved in an independent direction with the intention of augmenting both with additional features. How a component's sources have affected its construction will be explained in its respective section.

GRAD Machine

Overview

The GRAD Machine (hereby referred to as GMachine) is a von Neumann architecture computer. Its byte and word size are both 16 bits, denoting the size of a unit of memory and a CPU register respectively. As a byte and word are equal in size, the concept of endianness does not apply to this architecture; a word can fit in a single unit of memory. The entire fetch-decode-execute loop is completed in one clock cycle, with no support for pipelining. Finally, negative numbers are represented using two's complement.

The GMachine is composed of the following parts in addition to the supporting circuitry that connects them: Memory, Arithmetic Logic Unit (ALU), ALU Decoder, JMP Decoder, Program Counter, Input Decoders, Registers, and the Instruction Decoder. Their functionality and implementation will be outlined later.

Usage

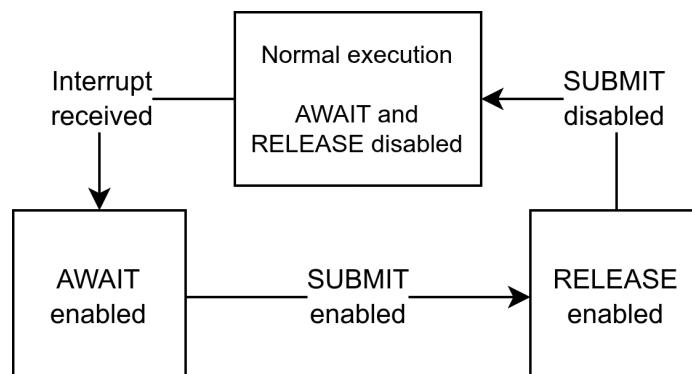
The GMachine implementation was constructed in the Digital^[2] circuit simulator. Running a program involves doing the following:

- Placing all of the .dig files within the same directory tree.
- Opening the CPU.dig file in the simulator.
- Placing the program file inside the current working directory, usually the location of the Digital executable.
- Renaming the program file to "program.c". The CPU.dig file is set up to automatically load any file with said name when it is ran, and reloading the program file requires only rerunning the circuit.

Although the notion of endianness does not apply to the CPU, as it always handles 16-bit chunks of data, compilation of the program file and being read afterwards by Digital are under the assumption that a byte is 8 bits. As such, a consistent endianness must be defined. The choice is arbitrary, with the CPU.dig file and the Grad Compiler having been preconfigured to Big-endian.

The GMachine is equipped with 4 input and 12 output "device" ports near the bottom of the circuit. There also lie the automatic and manual clocks, a button to toggle between them, as well as the reset button. Their functionality is as follows:

- The reset button can be used to restart program execution. Memory is not cleared from resets.
- The automatic clock is the default clock and its frequency can be changed.
- The manual clock can be used for per-instruction debugging, and is bound to the keyboard key K.
- The output ports are regular memory addresses that are transparent to the user.
- The input ports are used for receiving input from the user and their behaviour can be described from the following finite state diagram:



In short, the DEV AWAIT flag signifies to the user that the program is expecting data input, and the DEV SELECT output instructs the user on which input port it is expecting it from, with all other ports being ignored. When the user raises the SUBMIT flag to provide their input, DEV RELEASE is raised to instruct the user that the submit flag must now be lowered, after which:

- All flags are lowered.
- The user's input is finalised and received
- Program execution resumes.

Releasing the submit flag is necessary in order to guard against accidental future submissions. The mechanism can transition states only on clock pulses. Therefore, if the DEV SUBMIT flag is raised and lowered within the same clock cycle, no change is registered.

Functionality

Memory

The memory module of the GMachine acts as regular RAM for programs in addition to containing all the circuitry required for interacting with the device I/O ports. Its addressable range is 2^{15} , laid out as such:

- The first 2^{14} addresses are used as RAM.
- The next 4 addresses are used as device input ports.
- The 12 after are used as device output ports.
- The rest of the address space is unused.

Arithmetic Logic Unit

The ALU takes two 16-bit numbers (x and y), five 1-bit flags (negate x, negate y, negate output, zero x, zero y), and a 3-bit operation-circuit selector (f) as inputs. Combinations of these flags allow the ALU to perform more than one type of operation on some operation-circuits. The outputs of the ALU are the 16-bit output of the calculation, a signal indicating whether the output is negative (ng), and a signal indicating whether it is zero (zr).

Below follow the operations the ALU can compute, and which operation-circuit it uses or how it is derived from one. All unary operations use the value of x and ignore y. De Morgan's laws and the lemma $\neg x = \neg x + 1$ (1) are used for some proofs.

- Bitwise AND is calculated using a 16-bit AND gate.
- Bitwise OR is calculated using the AND circuit and DeMorgan's law, by negating x, y, and the output.

$$\neg(\neg x \& \neg y) \Leftrightarrow x | y$$
- Bitwise NOT is implemented using the AND circuit and negating x, and zeroing and negating y.

$$\neg x \& \neg FALSE \Leftrightarrow \neg x \& TRUE \Leftrightarrow \neg x$$
- Addition is implemented using a 16-bit Full Adder chain. Overflow is not handled.
- Subtraction is implemented using the addition circuit, and negating x and the output. The subtraction circuit subtracts the value of y from x. Similar to addition, underflow is not handled.

$$\begin{aligned} \neg(\neg x + y) &\Leftrightarrow (1) \\ \neg(-x - 1 + y) &\Leftrightarrow (1) \\ -(-x - 1 + y) - 1 &\Leftrightarrow x + 1 - y - 1 \Leftrightarrow x - y \end{aligned}$$

- Arithmetic negation is implemented using the addition circuit, zeroing and negating y, and negating the output.
 $\sim(x + \sim FALSE) \Leftrightarrow (1)$
 $-(x + \sim FALSE) - 1 \Leftrightarrow (1)$
 $-(x - FALSE - 1) - 1 \Leftrightarrow$
 $-x + FALSE + 1 - 1 \Leftrightarrow$
 $-x$
- Multiplication is implemented using a 16-bit multiplication circuit^[3]. Overflow is not handled.
- Integer division and modulo are implemented using a 16-bit unsigned division and modulo circuit^[4]. The outputs of the circuit are modified based on the signs of the inputs to allow for signed division, as defined by floored division^[6]. When dividing by zero the quotient will be -1 if x is non-negative, and 0 otherwise. The remainder will always be x.
- Absolute value is implemented with a custom circuit. The minimum number, -32768, is not handled, as it doesn't have a corresponding positive value. The circuit is implemented using the following formula:
 $\sim(\sim(x \wedge a) + a) \Leftrightarrow$, where a is a 16 bit long bit-string of the sign bit of x.
 If $x \geq 0$:
 $\sim(\sim(x \wedge 0) + 0) \Leftrightarrow$
 $\sim(\sim x) \Leftrightarrow$
 x
 If $x < 0$:
 $\sim(\sim(x \wedge a) + a) \Leftrightarrow$, a bit-string of ones represents -1 in two's complement.
 $\sim(\sim(x \wedge 0b1111111111111111) - 1) \Leftrightarrow$, XORing with a bit-string of ones is equivalent to the bitwise negation of the other member.
 $\sim(\sim x - 1) \Leftrightarrow$
 $\sim(x - 1) \Leftrightarrow (1)$
 $-(x - 1) - 1 \Leftrightarrow$
 $-x + 1 - 1 \Leftrightarrow$
 $-x$
- Square root is implemented with a 16-bit unsigned square root circuit^[5]. This circuit treats all inputs as 16-bit unsigned numbers as not to waste half the numerical range of 16 bits. This is safe to do as the square root of any number larger than 4 will always have half its original magnitude (and hence require one less bit to represent)*. Therefore, a 16-bit unsigned number's square root does not require the 16th bit, the sign bit, to be represented.
 To utilise the full range of the circuit, it is necessary to represent the input number as a signed number using two's complement. For a 16-bit number the mapping is as follows:
 If $x \in [0, 32767] \Rightarrow x$
 If $x \in [32768, 65535] \Rightarrow x - 2^{16}$

*Proof by induction:

Proposition: For any number $n \geq 4$: $\sqrt{n} \leq \frac{n}{2}$

Base case: For $n = 4$, $\sqrt{4} \leq \frac{4}{2}$ is true

Induction step: If the proposition holds for $k \geq 4$, then it should also hold for $k + 1$.

$$\sqrt{k+1} \leq \frac{k+1}{2} \Leftrightarrow (\sqrt{k+1})^2 \leq (\frac{k+1}{2})^2 \Leftrightarrow$$

$$k+1 \leq \frac{k^2+2k+1}{4} \Leftrightarrow 4k+4 \leq k^2+2k+1 \Leftrightarrow$$

$$3 \leq k^2 - 2k \Leftrightarrow, \text{ is true for } k \geq 4$$

- The NOP function returns the value of x. It is implemented by zeroing and inverting y, and then passing them through the AND circuit.
 $x \& \sim FALSE \Leftrightarrow$
 $x \& TRUE \Leftrightarrow$
 x
- The XOR function is implemented using a 16-bit XOR gate.

ALU Decoder

The ALU Decoder circuit is a lookup table that translates the specified ALU operation in an instruction to the corresponding ALU flags.

JMP Decoder

The JMP Decoder circuit is responsible for evaluating the jump condition from the current Instruction using the ng and zr outputs of the ALU. Due to the ALU's zr output, the GMachine can only perform comparisons with 0 as the right hand side.

Program Counter

The Program Counter circuit enables managing the instruction pointer through increments, jumps, and resets. The instruction pointer is stored as an unsigned 16-bit integer and, as such, has a maximum value of 65535.

Input Decoders

The Input Decoders use the current instruction to determine what the x and y inputs to the ALU should be.

Registers

The GMachine possesses four registers in total:

- Two constant registers with the values of 1 and 2 respectively. They help with optimisation by substituting for constants which would have to be loaded first.
- A general purpose D (Data) register. It is mainly used for storing intermediate calculations.
- The A (Address) register. While it can function as a general purpose register as well, its main purposes are threefold:
 - Load constants through A-instructions.
 - Function as a pointer to memory by storing the memory address to index.
 - Point to which instruction to jump to for a potential jump.
- Additionally, despite being a standalone component, memory is treated as the M register when used as an ALU input due to behaving similarly to one.

Instruction Decoder

The Instruction Decoder is responsible for determining the type of the current instruction and forwarding its values to other components.

Instruction definitions

The GRAD Machine has two types of instructions: A and C instructions, short for Address and Compute respectively. They are differentiated by the value of the highest bit in the instruction bit-string (instruction bit). An A-instruction always has a value of 0, whereas a C-instruction always has a value of 1.

A-instructions are used for loading constants into the A register, whether those constants represent memory addresses, instruction indexes, or numerical constants for computations. An A-instruction is defined as the instruction bit, followed by the 15-bit value to be loaded. As the highest bit of an A-instruction must be 0, and A-instructions being the only way to load constants into a program, a negative constant can never be loaded directly.

C-instructions specify to the CPU all the details necessary for running a computation. Namely, which inputs to use, which operation to run on those inputs, where to save the result, and an optional jump index and condition. Below is the specification of the C-instruction.

Bit index	Meaning	Comment
15	Instruction bit	Always 1 for C instructions
14	Memory bit	0 to interpret A register selection as A, 1 as M[A]
12-13	x register selection	Selection for the ALU's x input
10-11	y register selection	Similar to above
7-9	Result destination toggles	Bit toggles for where to save ALU result
4-6	Jump condition selection	Selects the jump condition
0-3	ALU operation selection	Selects the ALU operation

Jump condition selection	Condition
0, 0b000	No jump (False)
1, 0b001	Greater than (>)
2, 0b010	Equal (=)
3, 0b011	Greater equal ($\geq$)
4, 0b100	Less than (<)
5, 0b101	Not equal ($\neq$)
6, 0b110	Less equal ($\leq$)
7, 0b111	Jump (True)

Destination toggles	Destination
1, 0b001	M
2, 0b010	A
4, 0b100	D

Register selection	Register
0, 0b00	1
1, 0b01	2
2, 0b10	D
3, 0b11	A

Operation selection	Operation
00, 0b0000, 0x0	AND (&)
01, 0b0001, 0x1	OR ()
02, 0b0010, 0x2	NOT (~)
03, 0b0011, 0x3	ADD (+)
04, 0b0100, 0x4	SUB (−)
05, 0b0101, 0x5	NEG (−)
06, 0b0110, 0x6	MULT (*)
07, 0b0111, 0x7	DIV (/)
08, 0b1000, 0x8	MOD (%)
09, 0b1001, 0x9	ABS (x)
10, 0b1010, 0xA	SQRT (_/)
11, 0b1011, 0xB	NOP (None)
12, 0b1100, 0xC	XOR (^)

Implementation details

As the GMachine is an actual circuit rather than just a specification, there are some remarks to be made on the circuitry used for various parts.

Instruction decoder

The instruction decoder will output 0 from its C-instruction outputs when processing an A-instruction. This is done to ensure that components other than the A register are mutable only while processing a C-instruction.

Memory

In the memory module, of particular interest is the interrupt circuitry. For a program to raise an interrupt, two conditions must be met:

1. The Interruptor is tripped, which happens when the A register is set to one of the four device input port addresses.
2. The memory bit is set, which is possible only while processing a C-instruction due to the Instruction Decoder.

Should both of these conditions be satisfied, an interrupt is raised, lowering the INT OK (interrupt OK) flag. Without this flag the Program Counter will not increment itself and the rest of the GMachine will be blocked from receiving clock pulses until it is raised again.

An interrupt also enables the transaction mechanism for user input. Successfully submitting data will raise again the INT OK flag and execution will resume until the conditions mentioned above are met again.

It must also be mentioned that the device input ports have no capacity for retaining values and as such, ignore the flag to store results to memory.

Arithmetic Logic Unit

The ALU is a mostly straightforward circuit. Of note are how the ng and zr outputs are derived. The ng output utilises the two's complement representation of the GMachine and is enabled only if the highest bit of the result is 1. The zr output is a 16-way bitwise NOR of the result, as only 0 has no 1-bits in its binary representation.

JMP Decoder

The JMP Decoder is purely logical in function, signaling a jump if the following formula evaluates to true:

$$(NJ \wedge FALSE) \vee (GT \wedge \neg zr \wedge \neg ng) \vee (EQ \wedge zr) \vee (GE \wedge \neg ng) \vee (GT \wedge \neg zr \wedge ng) \vee (NE \wedge \neg zr) \vee (NE \wedge (ng \vee zr)) \vee (JMP \wedge TRUE)$$

With "NJ", "GT", "EQ" etc. being the logical representation of the selected jump condition. The hardware implementation uses Mux and DMux gates to decrease the size footprint of the circuit.

Program Counter

The Program Counter has three control inputs: increment, load, and reset. Although multiple can be enabled at once, only one will be acted upon, with reset having the highest priority, and increment the lowest. This ordering was chosen to allow increment to always be enabled (except during interrupts) without blocking jumps.

Input Decoder

The two Input Decoders select one of the registers each based on the memory bit and the register selection fields of the current C-instruction. Setting the memory bit will instruct both of the Input Decoders to use memory rather than the A register. Therefore it is not possible to use the A register and memory as inputs to the ALU simultaneously.

Registers

Of the registers, only one has special circuitry. The A register is wired to store its input value when either instructed so by a C-instruction, or when processing an A-instruction. Additionally, when processing an A-instruction, it will instead store its 15-bit value rather than the output of the ALU.

Usage of sources

The Elements of Computing Systems

The main source of knowledge for the construction of the GMachine was the book "The Elements of Computing Systems" by Noam Nisan and Shimon Schocken^[4]. Within it, the reader is tasked with constructing a 16-bit von Neumann processor (nicknamed the Hack machine) in HDL using only NAND gates as the foundational building block. The book assists the reader in three ways on this endeavor:

1. By explaining the importance and function of each chip that needs to be constructed.
2. By providing architectural guidelines, such as an instruction set, and the inputs and outputs of each chip.
3. By describing the operating logic of each chip through a set of equations that the HDL implementation must obey.

An initial partial implementation was created in HDL and then reimplemented in the environment of a circuit simulator, rendering both the HDL and simulator implementations original. The chip pinouts and instruction set, which were sourced from the book, have also undergone changes. These changes aim to increase the capabilities of the GMachine and are comprised of the following:

- The definition of the C-instruction was overhauled in order to accommodate the enhancements to the GMachine's ALU. The six comp bits and two unused bits were repurposed for the two register selection fields, using two bits each, and the operation selection field, utilising the remaining four bits.
- The ALU additionally supports multiplication, division, square root, bitwise XOR, and absolute value. Their addition at a hardware level allows for computing said operations in a single instruction rather than having to emulate them through software. Multiplication for example can be performed on the Hack Machine, but through repeated addition. Implementing this enhancement was trivial after overhauling the C-instruction's definition, requiring these changes:
 1. Widening the ALU's operation-circuit selector from one to three bits.
 2. Expanding the MUX gate, which selects which operation-circuit to use, from two to eight inputs.
 3. Developing or sourcing the operation-circuits, and connecting them to the output MUX gate.
- As a byproduct of changing the C-instruction's definition, the ALU can accept arbitrary combinations of registers and operations as inputs. In contrast, the Hack machine only allows combinations from a predefined list. The benefits to this approach are twofold:
 1. Debugging programs in the GMachine is easier, as the lookup tables are much smaller.
 2. Compiling GRAD Language code is easier, as combinations do not have to be checked for belonging to a predefined list.

Implementing this enhancement involved creating the ALU decoder and Input decoder circuits to handle the register and operation selector fields of a C-instruction. The ALU's flags are set by the ALU decoder, and, while it is not possible to select an arbitrary combination of flags, new presets can be defined.

- Although the Hack machine does provide I/O capabilities, it does so through its CPU simulator, meaning it lacks a hardware implementation. The GMachine has added all the necessary mechanisms for hardware-level I/O, also making the design manufacturable in theory.

Building Math Circuits

This is a series of blog posts by Coert Vonk^{[3][4][5]} in which he implements various math operations as digital circuits. The unsigned multiplication, division and square root circuits were sourced from here and subsequently tweaked to also handle signed numbers.

GRAD Language

Overview

The GRAD Language (hereby GLang) is a hybrid assembly/low-level language designed specifically for the GRAD Machine. Its hybrid nature means that some instructions have one-to-one equivalents to instructions in the GMachine (simple instructions), while others need to be decompressed into multiple assembly-level ones (complex instructions). The syntax of the language, as well as example programs can be found in the GRAD Language directory.

Grammar

A general outline of the language's grammar follows:

- Each line of source code may be empty, a comment, or a statement that is optionally followed by a comment.
- A statement is either a variable declaration, an alias declaration, a jump label, or an instruction. A variable is a memory location assigned to an identifier. An alias is similar but assigned a numeric constant or another alias instead.
- A jump label is an identifier used for jumping to a specific point in the program regardless of line number changes.
- There are three types of instructions:
 1. A-instructions, which map directly to the GMachine's A-instructions.
 2. C-instructions, that are further subdivided into simple and complex ones. Simple instructions similarly map directly to the GMachine's C-instructions, whereas complex ones correspond to multiple simple instructions and have to be decompressed before compilation. A C-instruction is considered complex if it uses any of the high-level features shown in [Programming in GRAD Language](#).
- All whitespace except for newlines is ignored.
- Lowercase text may be used only within comments. All language keywords are uppercase, and identifiers must be composed of uppercase letters, numbers, and the underscore character.

Programming in GRAD Language

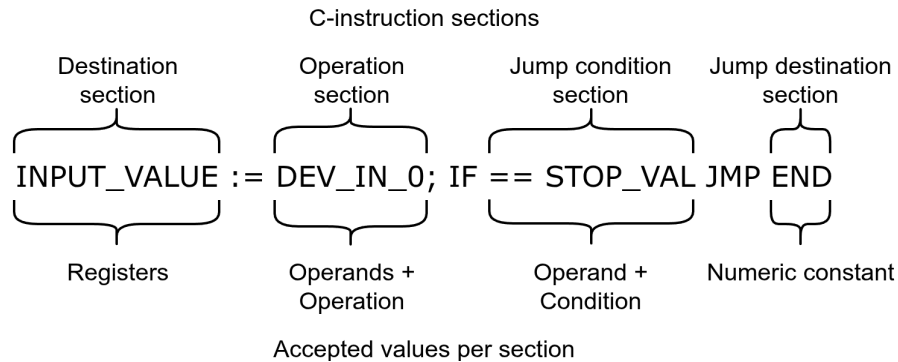
In this section the example program "showcase.g" will be used to explain the syntax of the GRAD Language, as well as show all of the high-level features in use.

The GLang mainly uses operands, which are composed of numeric constants, the registers (D, A, and memory as M), and identifiers. Identifiers represent either a numeric constant (aliases) or a memory location (variables). In either case they are equivalent to what they represent; a field which accepts only numeric constants does also accept aliases.

A set of identifiers are always defined for convenience when programming, in specific:

- M0 through M63 correspond to the first 64 memory locations.
- DEV_IN_0 through DEV_IN_3 correspond to the input device ports.
- DEV_OUT_0 through DEV_OUT_12 map to the twelve output device ports.
- DEV_IN_START_ADD and DEV_OUT_START_ADD equal the starting addresses of the input and output ports respectively. They can be used to iterate over them.

Each section of an instruction accepts different input types. A-instructions have one field and accept only numeric constants. C-instructions accept types based on the diagram below, featuring the 9th line of the showcase program. For reasons explained previously, the destination and jump destination sections also accept variables and aliases respectively.



The showcase program was designed with the intent of using as many of the language's complex features as possible. It operates as follows:

1. It reads a value from the user, if the value is equal to the stop value, it exits.
2. It reads a second value from the user.
3. Saves the first value to the memory address provided by the second value.
4. Repeats by returning to step 1.

An explanation of the program line by line follows:

- The third line is an alias declaration. It is assigning the numeric constant "0x275" to the identifier "STOP_VAL". As shown, numeric bases other than decimal are accepted as long as they are accompanied by their base specifier.
- The fifth line is a variable declaration. The identifier "INPUT_VALUE" is treated as a memory access to an address chosen by the compiler, usually one at random from the available memory pool.
- The seventh line is a jump label. It will be used in the fourth step to return to this point of the program.
- Line nine corresponds to the first step detailed above and also constitutes the first example of complex features. Assuming the jump label resolves to 0 and the compiler reserves the memory address 0x500 for INPUT_VALUE, the line transforms to:
M[0x500] := M[0x4000]; IF == 0x275 JMP 0

Here, all four complex features are used:

1. Inline memory indexing (M[0x500] and M[0x4000]).
2. Usage of constants other than 1 and 2 (0x275).
3. Comparison with a value other than 0 (IF == 0x275).
4. Inline definition of the jump destination (JMP 0).

Usage of complex features mark the instruction as complex, requiring the compiler to decompress it first. Decompression of the showcase program is shown in the GRAD Compiler's [Decompressing C-instructions](#) section. Decompressing a C-instruction heavily uses the D register. As such, using it in complex instructions is advised against, although not forbidden.

- At last, in line 11 (steps 2 to 4) of note is the destination part. "M[DEV_IN_1]" resolves to "M[M[0x4001]]". Indexing memory with another memory address' value is called recursive indexing and is considered an inline memory index.

Usage of sources

The Elements of Computing Systems

The GLang bases its grammar on the language proposed within the book, the Hack Machine language^[1]. Its differentiating factor is its expanded feature set, designed to make development for GMachine easier. The enhancements implemented are the following:

- Variables and aliases.
- Inline memory indexing (and subsequently recursive memory indexing).
- Comparison with values other than 0 in jump conditions.
- Special handling of the constants 1 and 2.
- Label instructions, which, although identical to the Hack Machine language's "Pseudo-commands", were developed independently.
- Additional comparison operands and operation symbols to support the added ALU capabilities.

GRAD Compiler

Overview

The GRAD Compiler (hereby GCompiler) is the software responsible for parsing, assembling, and compiling GLang source code into GMachine instructions. It is written in Python and is entirely original.

Usage

The GCompiler requires a Python interpreter of version 3.12 or higher to be installed. It is operated from the command line using the following syntax:

```
python -m grad_compiler <functions> <filename> [<filename>...]
```

Where:

- "filename" specifies which files or file paths the compiler should process.
- "functions" instructs the compiler whether to assemble, compile or do both to the specified files using the letters "a" and "c" respectively. Additionally, "h" can be used to print a short help menu.

Example:

```
python -m grad_compiler ac program.g
```

Compilation results are written to files in the "Output" directory with each step writing to a different file, differentiated by its file extension. Files with the .c extension are the output of the final step and can be executed in the GMachine. Files with the .p and .a file extensions are the results of the parser and assembler respectively and are used for debugging. Finally, the file extension chosen for GLang source code files is .g.

The GCompiler also offers some settings for changing its runtime behaviour in the "Options.py" file. Though most are intended for debugging purposes, the byte order can be changed here.

Compilation process

As a multi-pass compiler, the GCompiler processes each file in steps with each step processing its input line by line. After the compiler initialises, each file goes through the following stages:

1. Pre-processing, where whitespace and comments are removed. This is where the uppercase-only rule of the language is enforced by capitalising all text.
2. Parsing, during which statements are converted into an intermediate object representation.
3. Assembling, at which point:
 - C-instructions are decompressed.
 - Aliases are resolved to numeric constants.
 - Variable names are substituted with indexes to their assigned memory address.
 - Jump labels are replaced with line numbers.
 - Optimisations are applied.
4. Finally, compilation, during which the list of intermediate representation instructions is converted into GMachine instructions.

Implementation details

Parser

As the GLang is a relatively simple language, the parser module of the GCompiler opts to use delimiter symbols rather than the traditional recursive descent approach. The symbols used for parsing are not included in the identifier charset and therefore no ambiguity can arise during parsing. The sole exception to this rule is the delimiter between the jump destination and jump condition sections in a C-instruction, the literal string "JMP". Due to being composed exclusively of characters within the identifier charset, it must not appear as a substring in any identifier used in the jump part of a C-instruction.

Assembler

The assembler module of the GCompiler is responsible for processing GLang instructions in order to verify their validity and then simplify them for compilation. The two processes it performs that are of note are decompression and the optimisations it applies.

Decompression will be further explained in the next section. It involves managing the A register to load memory addresses and constants as they're needed, and manipulating jump conditions to always compare against 0.

Optimisations are applied in two steps. During decompression, the assembler tracks the state of the A register to avoid inserting A-instructions if the required value is already loaded. This is safe to do as all the ways the A register can be mutated are known and accounted for. After all instructions are decompressed, a set of optimisations targeting specific patterns are applied.

The final step of the assembler is resolving the used jump labels in C-instructions. This is performed last since all previous steps might change the number of instructions, requiring them to be fixed again in the end.

Compiler

Compilation is the final step of the GCompiler's pipeline. As the assembler can be skipped, some checks are performed again here in order to verify that the instructions supplied are valid.

Decompressing C-instructions

The GCompiler is responsible for decompressing complex C-instructions during assembling. Complex C-instructions are converted into simple ones before compiling those into machine code. Three lines from the showcase program require decompilation, line 9, 11, and 13. Decompression is done in steps with each step processing one instruction section. A C-instruction has its sections processed in the following order:

1. Operation section left operand.
2. Operation section right operand.
3. Destination section.
4. Jump condition section.
5. Jump destination section.

Starting with line 9, it reads:

```
INPUT_VALUE := DEV_IN_0; IF == STOP_VAL JMP END
```

Assuming the identifiers resolve to the same values as in [Programming in GRAD Language](#), the line translates to:

```
M[0x500] := M[0x4000]; IF == 0x275 JMP 0
```


Following the order outlined above, the operation section must be decompressed first. The section contains an inline memory index, requiring the address to be loaded before accessing memory.

```
@0x4000 // Load the address into the A register
```

```
M[0x500] := M; IF == 0x275 JMP 0 // M refers to the value stored in memory at the address loaded in the A register
```

The next section to be decompressed is the destination section, again for using an inline memory index. The destination section requires the A register to have a value that differs from the one already loaded. Therefore, it is necessary to temporarily store the value read from M[0x4000] somewhere. During decompression, the D register is used as intermediary storage when necessary, which occurs in two cases:

1. Two sections, or the two operands of the operation section, require different values to be loaded in the A register. This is the case for the operation and destination sections of this instruction.
2. A C-instruction that computes a binary operation also has a jump condition with a comparand other than 0. The intermediary store is required for manipulating the comparison.

```
@0x4000
```

```
D := M // Store the value of M[0x4000] in D
```

```
@0x500 // A changes, M now points to M[0x500]
```

```
M := D; IF == 0x275 JMP 0 // Store the previously assigned value of M[0x4000] to M
```

An instruction such as "M[123] := 123 + 123" which does not meet either of the criteria mentioned above does not need an intermediary store. However, it might require decompression still:

```
@123
```

```
M := A + A // A equals 123, M equals M[A]
```

The jump condition section requires two transformations to be applied. The comparison constant must be loaded, and the comparison must be manipulated to compare against 0. Again, the D register is used as intermediary storage as the value of the A register changes:

```
@0x4000
```

```
D := M
```

```
@0x500
```

```
M, D := D
```

```
@0x275 // Load the comparison constant
```

```
D := D - A; IF == 0 JMP 0 // If D == 0x275, then D - 0x275 == 0
```

It should be noted that only constants other than 1 or 2 need to be loaded. When using 1 or 2, the GMachine's constant registers can be used instead. For example the above would decompress to the following if it compared against 1:

```
...
```

```
M, D := D
```

```
D := D - 1; IF == 0 JMP 0 // Using the 1 register instead of having to load 1 as a constant
```

The final section to be decompressed is the jump destination section. As the value that needs to be loaded (0) is different to the current value of the A register (0x275), the D register is used again.

```
@0x4000
```

```
D := M
```

```
@0x500
```

```
M, D := D
```

```
@0x275
```

```
D := D - A // Store the manipulated comparison's left side
```

```
@0 // Load where to jump if the condition evaluates to true
```

```
D; IF == 0 JMP // Perform the comparison and jump
```

The next line that requires decompression is line 11. After resolving all its identifiers with the previous assumptions, it reads:

```
M[M[0x4001]] := M[0x500]; JMP 0
```

The first section to be decompressed is the operation section:

```
@0x500
```

```
D := M
```

```
M[M[0x4001]] := M; JMP 0
```

The second and most interesting section to be decompressed is the destination section, which contains a recursive memory index. It is decompressed as follows:

```
@0x500
```

```
D := M
```

```
@0x4001 // A = 0x4001, M = M[0x4001]
```

```
A = M // A = M[0x4001], M = M[M[0x4001]]
```

```
M = D; JMP 0
```

The jump condition is implicitly true, which doesn't require any decompression. Leaving only the jump destination section to be decompressed, resulting in:

```
@0x500
```

```
D := M
```

```
@0x4001
```

```
A = M
```

```
M = D
```

```
@0
```

```
D; JMP
```

The final line that needs to be decompressed is line 13, of which only the jump destination section uses a complex feature. Assuming the jump label resolves to 0, it reads:

```
D; JMP 0
```

Which then decompresses to:

```
@0
```

```
D; JMP
```

Sources

- [1] <https://www.nand2tetbookris.org/>, [Mirror](#)
- [2] <https://github.com/hneemann/Digital>, [Mirror](#)
- [3] <https://coertvonk.com/hw/building-math-circuits/faster-parameterized-multiplier-in-verilog-30774>, [Mirror](#)
- [4] <https://coertvonk.com/hw/building-math-circuits/parameterized-divider-in-verilog-30776>, [Mirror](#)
- [5] <https://coertvonk.com/hw/building-math-circuits/parameterized-square-root-in-verilog-30778>, [Mirror](#)
- [6] https://en.wikipedia.org/wiki/Modulo#Variants_of_the_definition